

KinetoStepTracker#

<https://pytorch.org/docs/stable/generated/torch.autograd.profiler.KinetoStepTracker.html>

Description:

Provides an abstraction for incrementing the step count globally. Previously, we only had one place to mark that a `step()` has occurred in the program via `pytorch profiler step()`. We will now add step hooks in the `Optimizer` class [pytorch/pytorch#88446](#). This could mean programs that already call `profiler.step()` every iteration can end up double incrementing step count. If a model uses multiple optimizers we can also have double or more counting of the step. We fix this by adding a layer of abstraction before calling `step()` to the `kineto` library. The idea is to maintain steps per requester in a dict:

Function Signature

```
class torch.autograd.profiler.KinetoStepTracker[source]#
```

```
classmethod current_step()[source]#
```

Return type

```
classmethod erase_step_count(requester)[source]#
```

Return type

Returns:

int

Code Examples

```
{
  "ProfilerStep": 100, # triggered by profiler step() call
  "Optimizer1Step": 100, # Optimizer 1 or 2 are just
examples, could be SGD, Adam etc
  "Optimizer2Step": 100,
}
```

```
{
  "ProfilerStep": 100,
  "Optimizer1Step": 101, # Optimizer1 got incremented first
say
  "Optimizer2Step": 100,
}
```

Detailed Documentation

Documentation

Provides an abstraction for incrementing the step count globally.

Previously, we only had one place to mark that a `step()` has occurred in the program via `pytorch profiler step()`. We will now add step hooks in the `Optimizer` class [pytorch/pytorch#88446](https://pytorch.org/docs/stable/pytorch.html#pytorch.profiler.step)

This could mean programs that already call `profiler.step()` every iteration can end up

double incrementing step count.

If a model uses multiple optimizers we can also have double or more counting of the step.

We fix this by adding a layer of abstraction before calling `step()` to the kineto library. The idea is to maintain steps per requester in a dict:

To figure out the global step count just take the max of dict values (100).

If one of the count increments the max will go up.

Then global step count is 101 We only call the kineto `step()` function when global count increments.

NOTE: Please do not use the `KinetoStepTracker` in modules beside the `Optimizer` for now. The result could be incorrect increments of the step count.

Get the latest step for any requester

Remove a given requester.

Increments the step count for the requester.

Additionally if the max over all step counts has incremented then trigger the `_kineto_step()` returns global step count

Initialize for a given requester.